task2_output.txt task3.py task3_output.txt task4.py task4_output.txt task5.py task5_output.txt task6.py task6_output.txt task7.py

```python
import os
import time
import numpy as np
from concurrent.futures import ThreadPoolExecutor

STUDENT_ID = "230103227"
ARRAY_SIZE = 30_000_000
REPEATS = 2
MAX_CORES = os.cpu_count() or 1
CORE_COUNTS = sorted(set([1, 2, MAX_CORES]))
def copy_chunk(args):
    source, destination, start, end = args
    np.copyto(destination[start:end], source[start:end])
def compute_chunk(args):
    source, destination, start, end = args
    x = source[start:end]
    y = destination[start:end]
    scratch = np.empty_like(x)
    np.sin(x, out=y)
    np.cos(x, out=scratch)
    y *= scratch
    np.tan(x, out=scratch)
    y += scratch
def run_kernel(source, destination, workers, operation):
    chunk_size = (ARRAY_SIZE + workers - 1) // workers
    tasks = [
        (source, destination, start, min(start + chunk_size, ARRAY_SIZE))
        for start in range(0, ARRAY_SIZE, chunk_size)
    ]
```

task2_output.txt task3.py task3_output.txt task4.py task4_output.txt task5.py task5_output.txt task6.py task6_output.txt task7.py

```python
def run_kernel(source, destination, workers, operation):
        for start in range(0, ARRAY_SIZE, chunk_size)
    ]
    start_time = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as executor:
        list(executor.map(operation, tasks))
    return time.perf_counter() - start_time
def benchmark(source, destination, workers, operation):
    timings = []
    for _ in range(REPEATS):
        elapsed = run_kernel(source, destination, workers, operation)
        timings.append(elapsed)
    return min(timings)
def main():
    print("TASK 7: ARITHMETIC INTENSITY VS MEMORY BOTTLENECKS")
    print(f"Student ID: {STUDENT_ID}")
    print(f"Array elements: {ARRAY_SIZE:,}")
    print("Data type: float32")
    print(f"Logical CPU cores: {MAX_CORES}")
    print(f"Tested worker counts: {CORE_COUNTS}")
    print(f"Runs per test: {REPEATS}")
    print()

    rng = np.random.default_rng(int(STUDENT_ID[-3:]))
    source = rng.random(ARRAY_SIZE, dtype=np.float32) * 0.5
    destination = np.empty_like(source)

    kernels = [
        ("Kernel A - Memory-Bound", copy_chunk),
```

task2_output.txt task3.py task3_output.txt task4.py task4_output.txt task5.py task5_output.txt task6.py task6_output.txt task7.py

```python
def main():
        ("Kernel A - Memory-Bound", copy_chunk),
        ("Kernel B - Compute-Bound", compute_chunk),
    ]

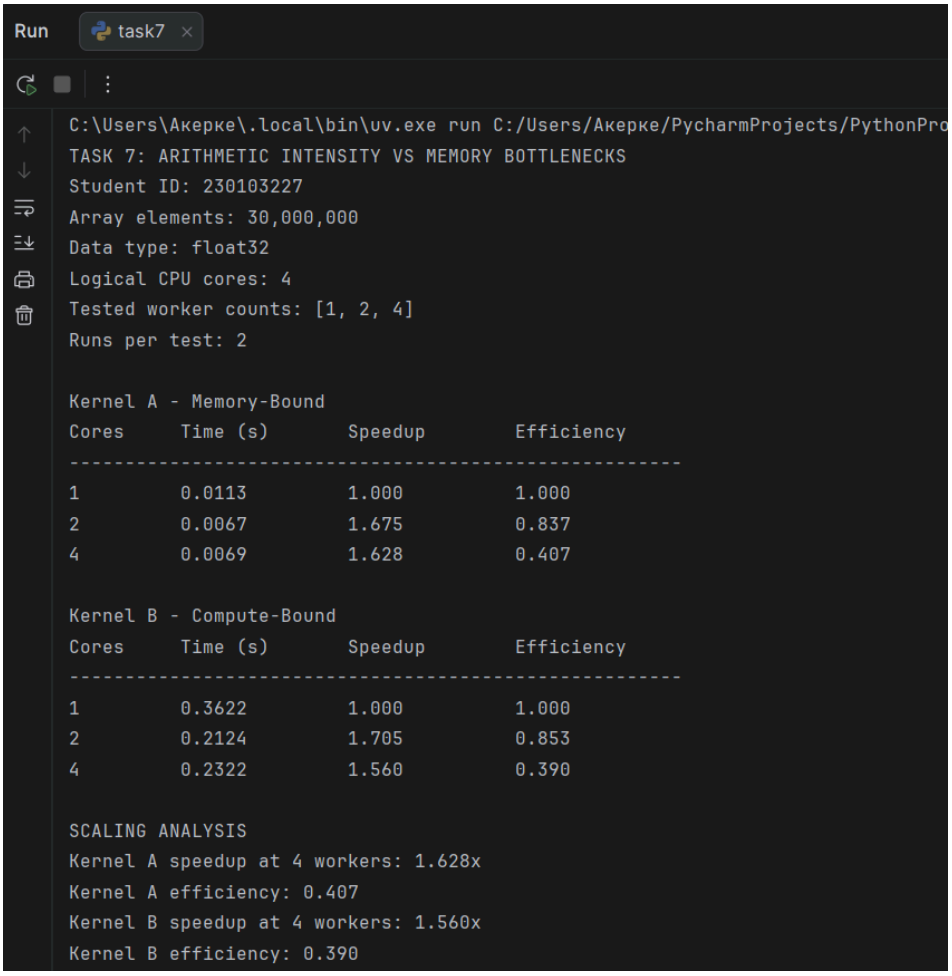
    all_results = {}
    for kernel_name, operation in kernels:
        print(kernel_name)
        print(
            f"{'Cores':<10}{'Time (s)':<15}"
            f"{'Speedup':<15}{'Efficiency':<15}"
        )
        print("-" * 55)
        kernel_results = {}
        for workers in CORE_COUNTS:
            elapsed = benchmark(source, destination, workers, operation)
            kernel_results[workers] = elapsed
        baseline = kernel_results[1]
        for workers in CORE_COUNTS:
            elapsed = kernel_results[workers]
            speedup = baseline / elapsed
            efficiency = speedup / workers
            print(
                f"{workers:<10}{elapsed:<15.4f}"
                f"{speedup:<15.3f}{efficiency:<15.3f}"
            )

        all_results[kernel_name] = kernel_results
        print()
```

task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py | task5_output.txt | task6.py | task6_output.txt | task7.py

```
40  def main():
82          print()
83
84      memory_results = all_results["Kernel A - Memory-Bound"]
85      compute_results = all_results["Kernel B - Compute-Bound"]
86      print("SCALING ANALYSIS")
87      if MAX_CORES > 1:
88          memory_speedup = memory_results[1] / memory_results[MAX_CORES]
89          compute_speedup = compute_results[1] / compute_results[MAX_CORES]
90          memory_efficiency = memory_speedup / MAX_CORES
91          compute_efficiency = compute_speedup / MAX_CORES
92          print(f"Kernel A speedup at {MAX_CORES} workers: {memory_speedup:.3f}x")
93          print(f"Kernel A efficiency: {memory_efficiency:.3f}")
94          print(f"Kernel B speedup at {MAX_CORES} workers: {compute_speedup:.3f}x")
95          print(f"Kernel B efficiency: {compute_efficiency:.3f}")
96          if memory_efficiency < compute_efficiency:
97              print(
98                  "Observed result: Kernel A scaled less efficiently than "
99                  "Kernel B. This is consistent with memory-bandwidth "
100                 "saturation, but timings alone cannot prove it."
101             )
102         elif memory_speedup < compute_speedup:
103             print(
104                 "Observed result: Kernel A gained less speedup than "
105                 "Kernel B, consistent with a memory-bandwidth bottleneck."
106             )
107         else:
108             print(
109                 "Observed result: Kernel A did not show a stronger "
```

```
104                 "Observed result: Kernel A gained less speedup than "
105                 "Kernel B, consistent with a memory-bandwidth bottleneck."
106             )
107         else:
108             print(
109                 "Observed result: Kernel A did not show a stronger "
110                 "scaling bottleneck than Kernel B in this run."
111             )
112
113     print()
114     print(
115         "Note: Efficiency = Speedup / Cores. Results can vary with "
116         "CPU, RAM bandwidth, background processes, and thread scheduling."
117     )
118
119
120 if __name__ == "__main__":
121     main()
```



task3.py task3_output.txt task4.py task4_output.txt task5.py task5_output.txt task6.py task6_output.txt task7.py task7_output.txt

```
TASK 7: ARITHMETIC INTENSITY VS MEMORY BOTTLENECKS
Student ID: 230103227
Array elements: 30,000,000
Data type: float32
Logical CPU cores: 4
Tested worker counts: [1, 2, 4]
Runs per test: 2

Kernel A - Memory-Bound
Cores     Time (s)       Speedup        Efficiency
------------------------------------------------------
1         0.0113         1.000          1.000
2         0.0067         1.675          0.837
4         0.0069         1.628          0.407

Kernel B - Compute-Bound
Cores     Time (s)       Speedup        Efficiency
------------------------------------------------------
1         0.3622         1.000          1.000
2         0.2124         1.705          0.853
4         0.2322         1.560          0.390

SCALING ANALYSIS
Kernel A speedup at 4 workers: 1.628x
Kernel A efficiency: 0.407
Kernel B speedup at 4 workers: 1.560x
Kernel B efficiency: 0.390
Observed result: Kernel A did not show a stronger scaling bottleneck than Kernel B in this run.

Note: Efficiency = Speedup / Cores. Results can vary with CPU, RAM bandwidth, background processes, and thread scheduling.
```